

HomeWork2

Aswath Oruganti

June 7, 2026

1 Concept Questions

1. First principle component proof

1.1 The objection function: Simplified

The variance of the proejcted data is provided below in the objective function. Starting with this objective function below, we will derive the equivalent optimization problem that defines the first principal component direction:

$$\max_{w: \|w\| \leq 1} \frac{1}{m} \sum_{i=1}^m (w^T x^i - w^T \mu)^2$$

As a first step, we factor out the weight vector w^T from the term inside the square:

$$w^T x^i - w^T \mu = w^T (x^i - \mu)$$

The above expression is substituted into the squared term. Because the dot product results in a scalar, we can rewrite the square of the scalar as the product of the term and its transpose:

$$(w^T (x^i - \mu))^2 = (w^T (x^i - \mu)) (w^T (x^i - \mu))^T$$

Using the property of transposes $(AB)^T = B^T A^T$, the second term becomes $(x^i - \mu)^T w$. Substituting this back yields:

$$(w^T (x^i - \mu)) ((x^i - \mu)^T w) = w^T (x^i - \mu) (x^i - \mu)^T w$$

As a next step, plug this expanded form back into the original summation:

$$\frac{1}{m} \sum_{i=1}^m [w^T (x^i - \mu) (x^i - \mu)^T w]$$

Since w is independent of the summation index i , we can pull w^T and w outside the summation:

$$w^T \left[\frac{1}{m} \sum_{i=1}^m (x^i - \mu) (x^i - \mu)^T \right] w$$

The term enclosed in the brackets is the exact definition of the sample covariance matrix, denoted as Σ . Therefore, the objective function simplifies beautifully to:

$$\max_{w: \|w\| \leq 1} w^T \Sigma w$$

1.2 The constrained optimizer: Setup

The maximization of $w^T \Sigma w$ is subjected to the constraint $\|w\| \leq 1$. The positive semi-definite nature of the matrix Σ indicates that the maximum variance will always occur on the boundary of the constraint sphere. The constraint hence can be treated as an equality equation: $\|w\|^2 = w^T w = 1$.

As learned in the lectures, we can use the method of Lagrange multipliers to solve this constrained maximization problem. So by adding the multiplier λ we set up the Lagrangian function $\mathcal{L}$:

$$\mathcal{L}(w, \lambda) = w^T \Sigma w - \lambda(w^T w - 1)$$

1.3 Finding the maximum

To find the maximum, we take the partial derivative (gradient) of the Lagrangian with respect to the vector w and set it to zero:

$$\nabla_w \mathcal{L} = 2\Sigma w - 2\lambda w = 0$$

Dividing both sides by 2 and rearranging gives:

$$\Sigma w = \lambda w$$

We have established that the optimal direction w is an eigenvector. Now we must determine *which* eigenvector maximizes our variance.

Plugging back the eigenvector identity ($\Sigma w = \lambda w$) back into our simplified objective function from Step 1.1:

$$w^T \Sigma w = w^T (\lambda w) = \lambda(w^T w)$$

Since our initial constraint established that $w^T w = 1$, the expression simplifies completely:

$$\lambda(1) = \lambda$$

This shows that the variance of the data projected onto any eigenvector w is exactly equal to its corresponding eigenvalue λ .

It also establishes the fact that we must define the first principal component direction v as the eigenvector w that corresponds to the **largest eigenvalue** $\lambda_{\max}$ of the covariance matrix Σ in order to maximize the variance.

$$v = \arg \max_w (w^T \Sigma w) \implies \Sigma v = \lambda_{\max} v$$

2. Eigenvalue decomposition of A

We can use the mathematical process for finding both eigenvalues and their corresponding eigenvectors. The process can be built around solving the fundamental eigenvalue equation:

$$Av = \lambda v$$

Where:

- A is your square matrix.
- v is the eigenvector (a non-zero vector).
- λ is the eigenvalue (a scalar number).

Providing a mathematical way to find the eigenvalues and eigenvectors of A involves three main steps:

We start by rearranging the fundamental equation so all the terms are on one side:

$$Av - \lambda v = 0$$

To factor out the vector v , we must multiply the scalar λ by the Identity matrix I (so we are subtracting a matrix from a matrix, rather than a number from a matrix):

$$(A - \lambda I)v = 0$$

For this equation to have a valid solution (where v is not just a vector of all zeros), the matrix $(A - \lambda I)$ must squish space down in such a way that it is not invertible. In linear algebra, a matrix is non-invertible if and only if its determinant is exactly zero.

This gives us the **Characteristic Equation**:

$$\det(A - \lambda I) = 0$$

After this we calculate the determinant of $(A - \lambda I)$ and set it equal to zero. This will always create a polynomial equation (for a 2×2 matrix, it will be a quadratic equation; for a 3×3 , a cubic equation, etc.).

Solving for the roots of this polynomial gives you your eigenvalues $(\lambda_1, \lambda_2, \dots)$.

Once you have your eigenvalues, you find the eigenvectors by plugging each λ back into the factored equation:

$$(A - \lambda I)v = 0$$

For each eigenvalue, this will create a system of linear equations. You solve this system (often using row reduction or simple substitution) to find the components of the vector v .

Below is the solution for our specific diagonal matrix to find the eigenvalues and eigenvectors:

$$A = \begin{bmatrix} 4 & 0 \\ 0 & 3 \end{bmatrix}$$

2.1 Characteristic Equation

We start by finding the determinant of $(A - \lambda I)$ and setting it to zero:

$$\begin{aligned} \det \left(\begin{bmatrix} 4 & 0 \\ 0 & 3 \end{bmatrix} - \begin{bmatrix} \lambda & 0 \\ 0 & \lambda \end{bmatrix} \right) &= 0 \\ \det \left(\begin{bmatrix} 4 - \lambda & 0 \\ 0 & 3 - \lambda \end{bmatrix} \right) &= 0 \end{aligned}$$

Calculate the determinant ($ad - bc$):

$$(4 - \lambda)(3 - \lambda) - (0)(0) = 0$$

2.2 Solve for the Eigenvalues (λ)

Because the matrix was already diagonal, the determinant simplifies immediately to its factored form:

$$(4 - \lambda)(3 - \lambda) = 0$$

Setting each factor to zero gives us our two eigenvalues:

$$\begin{aligned} \lambda_1 &= 4 \\ \lambda_2 &= 3 \end{aligned}$$

2.3 Solve for the Eigenvectors (v)

To find the eigenvector for $\lambda_1 = 4$, we substitute $\lambda = 4$ back into the equation $(A - \lambda I)v = 0$. To find the eigenvector for $\lambda_2 = 3$, we substitute $\lambda = 3$ back into the same equation.

Substitute $\lambda = 4$ into the equation $(A - \lambda I)v = 0$:

$$\begin{bmatrix} 4-4 & 0 \\ 0 & 3-4 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$
$$\begin{bmatrix} 0 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

This translates into a system of linear equations:

- $0x + 0y = 0 \implies$ This is true for any value of x . Therefore, x is a **free variable**.
- $0x - 1y = 0 \implies -y = 0 \implies y = 0$.

Since x can be anything, but y must strictly be 0, we pick the simplest non-zero number for x (which is 1). This gives us our first eigenvector:

$$v_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Substitute $\lambda = 3$ into the equation $(A - \lambda I)v = 0$:

$$\begin{bmatrix} 4-3 & 0 \\ 0 & 3-3 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$
$$\begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

This translates into a system of linear equations:

- $1x + 0y = 0 \implies x = 0$.
- $0x + 0y = 0 \implies$ This is true for any value of y . Therefore, y is a **free variable**.

Since y can be anything, but x must strictly be 0, we pick the simplest non-zero number for y (which is 1). This gives us our second eigenvector:

$$v_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

2.4 Decompositions ($A = Q\Lambda Q^{-1}$)

Now that we have derived the eigenvalues and eigenvectors mathematically, we can assemble the matrices.

The diagonal matrix Λ is formed by placing the eigenvalues on the diagonal. The matrix Q is formed by placing the corresponding eigenvectors as its columns.

Decomposition 1: For the first decomposition, we place the eigenvalues in Λ in their original order.

$$\Lambda = \begin{bmatrix} 4 & 0 \\ 0 & 3 \end{bmatrix}$$
$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Because Q is the Identity matrix (I), its inverse is simply itself:

$$Q^{-1} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Mathematical Proof of Validity: We must prove that $Q\Lambda Q^{-1} = A$.

$$Q\Lambda Q^{-1} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 4 & 0 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Multiplying the first two matrices:

$$= \begin{bmatrix} 4 & 0 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Multiplying by the inverse:

$$= \begin{bmatrix} 4 & 0 \\ 0 & 3 \end{bmatrix} = A$$

This proves the first decomposition is valid.

Decomposition 2: For the second distinct decomposition, we will swap the order of the eigenvalues in Λ . Consequently, we must swap the order of the corresponding eigenvectors (columns) in Q .

$$\Lambda = \begin{bmatrix} 3 & 0 \\ 0 & 4 \end{bmatrix}$$

$$Q = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

To find Q^{-1} , we can either compute it manually or recognize that Q is a standard permutation matrix (which is its own inverse, meaning $QQ = I$).

$$Q^{-1} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Mathematical Proof of Validity: We must prove that $Q\Lambda Q^{-1} = A$.

$$Q\Lambda Q^{-1} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 3 & 0 \\ 0 & 4 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

First, multiply Q and Λ :

$$\begin{bmatrix} (0)(3) + (1)(0) & (0)(0) + (1)(4) \\ (1)(3) + (0)(0) & (1)(0) + (0)(4) \end{bmatrix} = \begin{bmatrix} 0 & 4 \\ 3 & 0 \end{bmatrix}$$

Now, multiply that intermediate result by Q^{-1} :

$$\begin{bmatrix} 0 & 4 \\ 3 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} (0)(0) + (4)(1) & (0)(1) + (4)(0) \\ (3)(0) + (0)(1) & (3)(1) + (0)(0) \end{bmatrix}$$

Simplifying the arithmetic gives us the final matrix:

$$= \begin{bmatrix} 4 & 0 \\ 0 & 3 \end{bmatrix} = A$$

This proves the second distinct decomposition is mathematically valid, successfully demonstrating that A can be decomposed in multiple valid ways.

2 Graph Laplacian and GNN

1. Neighborhood aggregation

1.1 1-Hop Neighbors

To Show that $(Lx)_i = d_i x_i - \sum_{j \sim i} x_j$, we use the mathematical definition of the Graph Laplacian $L = D - A$. Let x be a feature vector containing a value for each node. When we multiply the matrix L by the vector x , the i^{th} element of the resulting vector is simply the dot product of the i^{th} row of L with the vector x . So, we can write the expression as below by summing over j :

$$(Lx)_i = \sum_{j=1}^n L_{ij} x_j$$

we then substitute the definition of the entries of L . We look at the L (laplacian) and by observation derive some properties of its entries as below:

- **Diagonal entries** ($i = j$): $L_{ii} = d_i$ (the degree of node i).
- **Off-diagonal entries** ($i \neq j$): $L_{ij} = -1$ if node j is a neighbor of node i (denoted as $j \sim i$), and 0 if they are not connected.

Splitting the summation into the diagonal term and the off-diagonal terms gives us the below simplification. This is due to the fact that the non-diagonal ($i \sim j$) entries resolve it to -1:

$$\begin{aligned}(Lx)_i &= L_{ii} x_i + \sum_{j \neq i} L_{ij} x_j \\(Lx)_i &= d_i x_i + \sum_{j \sim i} (-1) x_j \\(Lx)_i &= d_i x_i - \sum_{j \sim i} x_j\end{aligned}$$

Conclusion: This proves that multiplying by L updates node i 's feature by taking its own degree-weighted feature and subtracting the sum of the features of its immediate 1-hop neighbors.

1.2 2-Hop Neighbors

In order to understand why $L^2 x$ includes terms that traverse paths up to 2-hops, think of $L^2 x$ as applying the Laplacian twice: $L^2 x = L(Lx)$. Let's define a new vector $y = Lx$. From part (2.1.1.1), we know that y_i contains mixed information from node i and its 1-hop neighbors.

Now apply L a second time to this new vector y :

$$(L^2 x)_i = (Ly)_i = d_i y_i - \sum_{j \sim i} y_j$$

Substitute y back to its original Lx form:

$$(L^2 x)_i = d_i (Lx)_i - \sum_{j \sim i} (Lx)_j$$

If we take a closer look at the terms in this expression, we can see how information from 2-hop neighbors is being aggregated:

- $(Lx)_i$ contains information from node i 's 1-hop neighbors.
- $(Lx)_j$ contains information from node j 's 1-hop neighbors. Because we are summing over $j \sim i$ (meaning node j is a 1-hop neighbor of i), pulling in information from j 's 1-hop neighbors means we are pulling in information from nodes that are $1 + 1 = 2$ hops away from node i . Thus, information has propagated exactly 2 hops.

1.3 Multi-Hop Neighbors

In order to come to the conclusion on why applying a polynomial in L aggregates multi-hop information, let's look at a polynomial in L of order K take the form: $P(L) = \theta_0 I + \theta_1 L + \theta_2 L^2 + \dots + \theta_K L^K$.

When we apply this to a feature vector x :

$$P(L)x = \theta_0 x + \theta_1 (Lx) + \theta_2 (L^2 x) + \dots + \theta_K (L^K x)$$

Based on the pattern established in (2.1.1.1) and (2.1.1.2):

- x contains 0-hop info (just the node itself).
- Lx contains up to 1-hop info.
- $L^2 x$ contains up to 2-hop info.
- $L^K x$ contains up to K -hop info.

Therefore, applying a polynomial in L effectively aggregates information from the node's entire local neighborhood up to a K -hop radius "in one shot", beautifully weighting each hop distance by the learnable coefficients θ_k .

2. Small Example Calculation

2.1 Explicit Matrix Calculation

We are given the 3-node path adjacency matrix A and told to compute $T_2(L) = 2L^2 - I$.

2.1.1 Verify $L = D - A$

From A , node 1 has 1 connection, node 2 has 2 connections, and node 3 has 1 connection.

$$D = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad A = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

$$L = D - A = \begin{bmatrix} 1 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 1 \end{bmatrix}$$

2.1.2 Compute L^2

Multiply L by itself:

$$L^2 = \begin{bmatrix} 1 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 1 \end{bmatrix} \begin{bmatrix} 1 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 1 \end{bmatrix}$$

- **Row 1, Col 1:** $(1)(1) + (-1)(-1) + (0)(0) = 2$
- **Row 1, Col 2:** $(1)(-1) + (-1)(2) + (0)(-1) = -3$
- **Row 1, Col 3:** $(1)(0) + (-1)(-1) + (0)(1) = 1$
- **Row 2, Col 1:** $(-1)(1) + (2)(-1) + (-1)(0) = -3$
- **Row 2, Col 2:** $(-1)(-1) + (2)(2) + (-1)(-1) = 6$
- **Row 2, Col 3:** $(-1)(0) + (2)(-1) + (-1)(1) = -3$
- **Row 3, Col 1:** $(0)(1) + (-1)(-1) + (1)(0) = 1$
- **Row 3, Col 2:** $(0)(-1) + (-1)(2) + (1)(-1) = -3$
- **Row 3, Col 3:** $(0)(0) + (-1)(-1) + (1)(1) = 2$

$$L^2 = \begin{bmatrix} 2 & -3 & 1 \\ -3 & 6 & -3 \\ 1 & -3 & 2 \end{bmatrix}$$

2.1.3 Compute $T_2(L) = 2L^2 - I$

First, multiply L^2 by 2:

$$2L^2 = \begin{bmatrix} 4 & -6 & 2 \\ -6 & 12 & -6 \\ 2 & -6 & 4 \end{bmatrix}$$

Now, subtract the Identity matrix I :

$$T_2(L) = \begin{bmatrix} 4 & -6 & 2 \\ -6 & 12 & -6 \\ 2 & -6 & 4 \end{bmatrix} - \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Final Explicit Matrix:

$$T_2(L) = \begin{bmatrix} 3 & -6 & 2 \\ -6 & 11 & -6 \\ 2 & -6 & 3 \end{bmatrix}$$

3 Order of faces using ISOMAP

1. Nearest Neighbor Graph Visualization

1.1 Methodology:

To construct the ϵ -ISOMAP nearest neighbor graph, pairwise Euclidean distances were computed across all 698 images in the $\mathbb{R}^{4096}$ dimensional space. An adjacency matrix A was formed by applying a distance threshold (ϵ). If the distance between two images x^i and x^j was less than or equal to ϵ , the true distance was retained as an edge weight; otherwise, the edge was set to 0, indicating no direct connection.

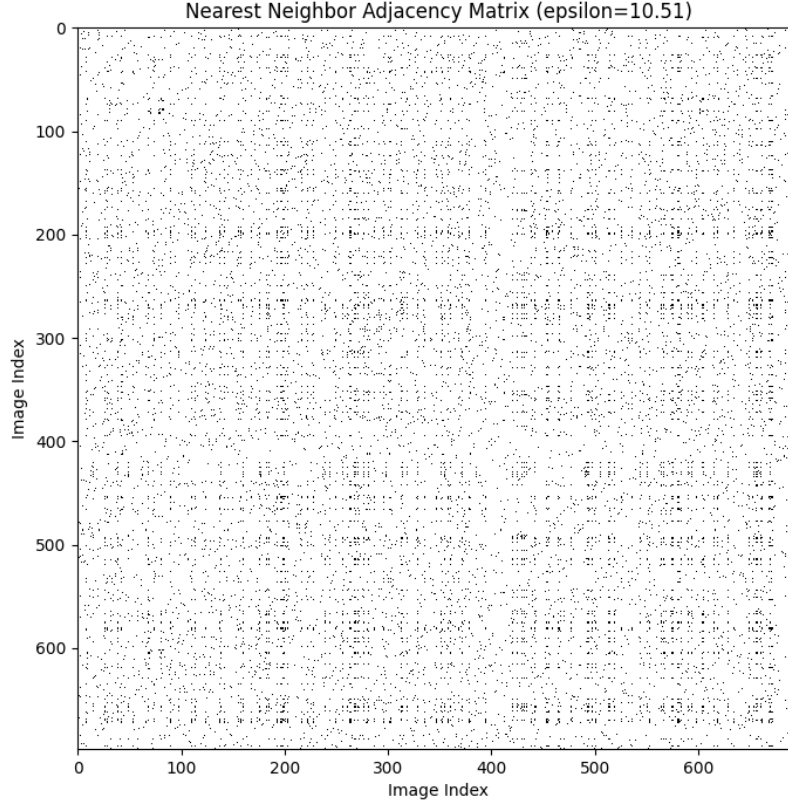


Figure 1: Nearest Neighbor Adjacency Matrix

1.2 Observations:

The resulting sparse adjacency matrix (visualized above) displays a block-diagonal-like structure with scattered off-diagonal elements. This indicates that while the dataset contains sequences of highly similar, temporally or spatially adjacent frames (the dense blocks), there are also complex looping structures in the high-dimensional space where disparate parts of the sequence come close together.

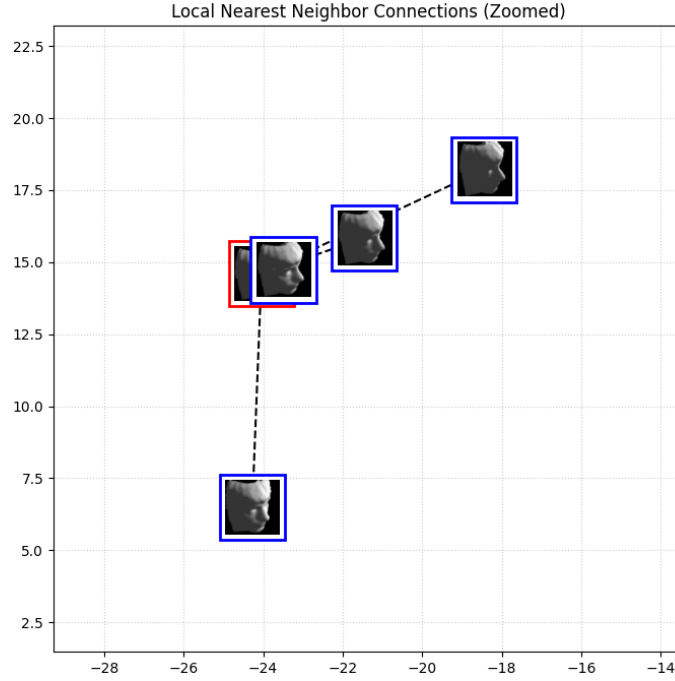


Figure 2: Local Nearest Neighbor Connections (Zoomed subgraph with specific face nodes)

2. ISOMAP Implementation & Manifold Analysis

2.1 Tuning the Kernel Bandwidth (ϵ):

A critical challenge in ϵ -ISOMAP is selecting an appropriate kernel bandwidth. If ϵ is too large, the algorithm "short-circuits" the manifold, essentially devolving into standard PCA. If ϵ is too small, the graph fractures into disconnected components, making it impossible to compute a global shortest-path matrix D . To objectively tune this parameter, the Minimum Spanning Tree (MST) of the fully dense Euclidean distance matrix was computed. The MST represents the minimum set of edges required to keep all 698 nodes connected in a single graph. By finding the *maximum* edge weight within this MST, we isolate the exact mathematical lower bound for ϵ . Setting ϵ just slightly above this maximum MST edge (e.g., adding a 1% buffer) guarantees that the neighborhood graph is fully connected while strictly minimizing the risk of non-linear short-circuiting across empty space.

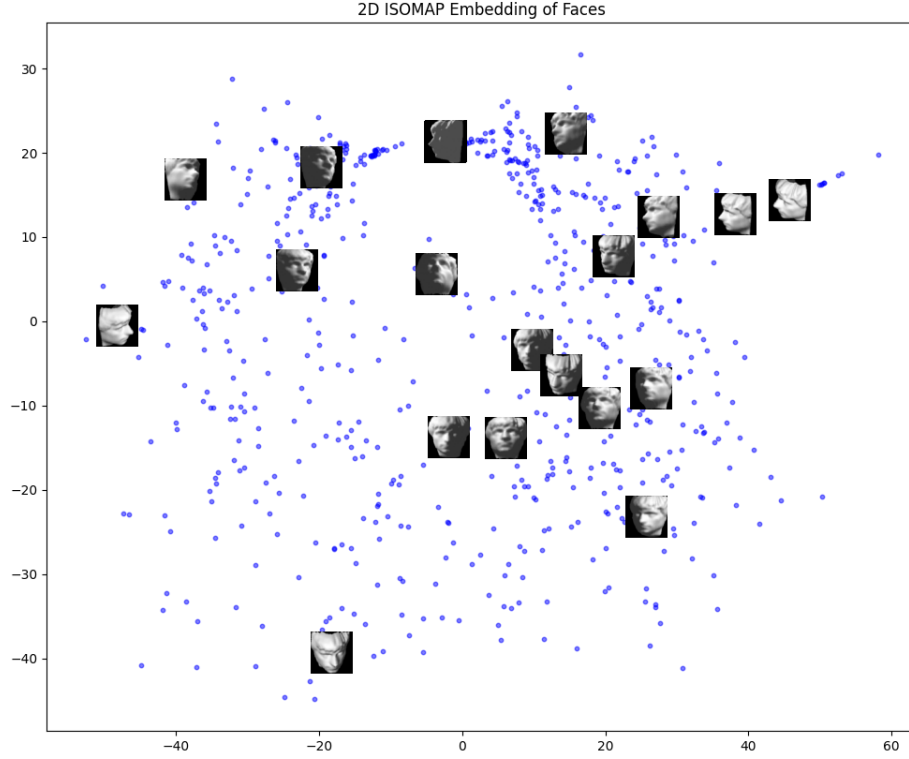


Figure 3: 2D ISOMAP Embedding of Faces

2.2 Visual Similarity & Arrangement Analysis:

The 2D embedding successfully unrolls the high-dimensional data, revealing a highly structured, meaningful coordinate system similar to the original Tenenbaum et al. (2000) paper.

- **The X-Axis (Yaw):** Moving horizontally across the embedding space corresponds smoothly to the left-to-right rotation of the face. Faces on the far left are looking sharply left, smoothly transitioning to front-facing in the center, and right-facing on the far right.
- **The Y-Axis (Pitch / Lighting):** Moving vertically across the embedding captures variations in vertical tilt (looking up vs. looking down) and lighting direction.
- **Conclusion:** The ISOMAP algorithm successfully discovered the intrinsic degrees of freedom (pose and lighting) governing the dataset. Neighboring points in this 2D space are highly visually similar, proving that geodesic distance computation accurately captured the continuous manifold of human movement.

3. PCA Projection & Method Comparison

3.1 Methodology:

For comparison, a standard Principal Component Analysis (PCA) was performed to project the $\mathbb{R}^{4096}$ data onto its top two principal eigenvectors, maximizing the retained variance using a purely linear transformation.

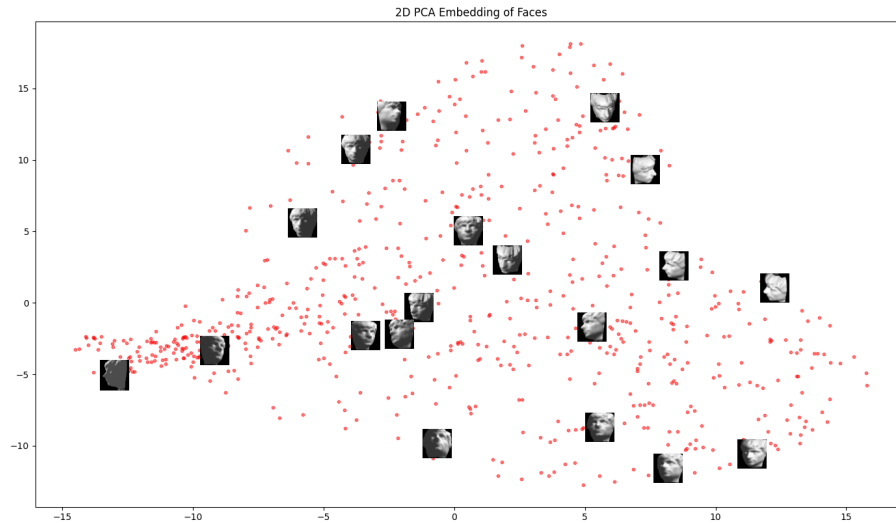


Figure 4: 2D PCA Embedding of Faces

3.2 Comparison: Is ISOMAP more meaningful than PCA?

Yes, the ISOMAP projection is significantly more meaningful and interpretable than the PCA projection for this dataset.

3.3 Explanation of PCA's Failure:

PCA relies strictly on global Euclidean distances (L_2 norm). It assumes that the data lies on or near a linear subspace. However, image transformations like 3D rotation and directional lighting are highly non-linear operations in pixel space. Because PCA measures straight lines directly through the high-dimensional space, it falsely registers points on opposite sides of the "curve" as being similar. Consequently, the PCA scatter plot appears highly cluttered and jumbled. Faces looking left, right, and center overlap near the origin, failing to isolate the distinct axes of movement.

3.4 Explanation of ISOMAP's Success:

ISOMAP succeeds because it replaces the global Euclidean distance with **geodesic distance** (shortest paths along the neighborhood graph). By strictly enforcing local Euclidean distances (where the manifold is approximately flat) and integrating them to find global distances, ISOMAP respects the actual geometry of the data cloud. It successfully "flattens" the curved surface of the face poses, yielding a neat, disentangled 2D representation that PCA is mathematically incapable of finding.

4 Eigenfaces & simple Face Recognition

1. PCA Analysis and Eigenface Visualization (10 points)

1.1 Methodology:

To perform the Principal Component Analysis (PCA), the training images for Subject 1 and Subject 2 were isolated (excluding `subject01-test.gif` and `subject02-test.gif`). Each grayscale image was downsampled by a factor of 4 using bilinear interpolation to reduce the computational load, and then flattened into a

1D vector. Mean centering was applied to the training sets, followed by an eigendecomposition of the covariance matrix to extract the principal components (eigenvectors). The top 6 eigenvectors corresponding to the largest eigenvalues were retained and reshaped back into the original 2D image dimensions for visualization.

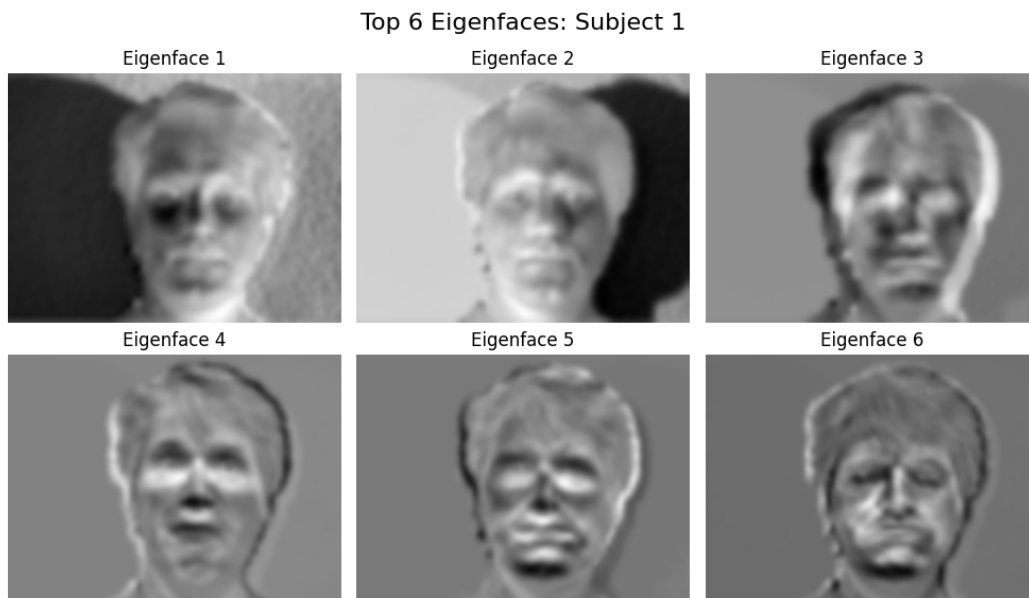


Figure 5: Top 6 Eigenfaces for Subject 1

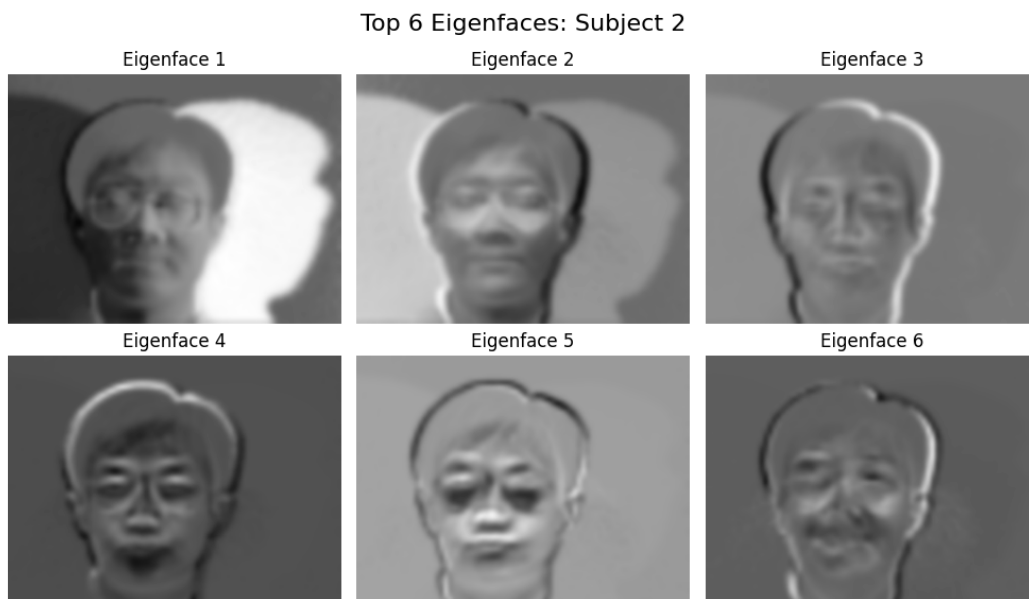


Figure 6: Top 6 Eigenfaces for Subject 2

1.2 Visual Patterns in the Top 6 Eigenfaces:

When visualizing the top 6 eigenfaces, they appear as blurry, ghostly face-like structures. Because PCA mathematically extracts the directions of maximum variance in the dataset, these images do not represent actual photos of the subject, but rather the most prominent ways in which the subject's face changes across the different photos. A clear pattern emerges as we move from the 1st to the 6th eigenface:

- **Eigenfaces 1 and 2 (Dominant Patterns):** These capture massive, global variations, primarily **lighting changes and broad shadows**. For instance, one side of the face might be bright white while the other is pitch black, indicating that a strong directional light source moving across the room was the largest source of variance in the raw pixel data.
- **Eigenfaces 3 and 4:** These capture secondary, broader structural changes, such as the subject tilting their head up or down, or light shining from above versus below (creating shadows under the nose and eyebrows).
- **Eigenfaces 5 and 6:** As we move to lower-variance components, the patterns become higher-frequency and more localized. Instead of broad shadows, these eigenfaces highlight specific facial features that changed, such as the mouth opening (expressions), the eyes squinting, or the presence of reflections on the subject's glasses.

2. Face Recognition and Projection Residuals (10 points)

2.1 Methodology and Scores:

To perform the face recognition task, the two test images were downsampled, vectorized, and mean-centered using the respective training means. The projection residual s_{ij} was calculated to measure the squared Euclidean distance between the test image and its reconstruction from the 6-dimensional eigenface subspace. The formula used is:

$$s_{ij} = \|(\text{test image})_j - (\text{eigenface}_i)(\text{eigenface}_i)^T(\text{test image})_j\|_2^2$$

The resulting scores from the algorithm execution are reported below:

	Test Image 1 (subject01-test)	Test Image 2 (subject02-test)
Subject 1 Eigenfaces ($i = 1$)	$s_{11} = 5,251,744.37$	$s_{12} = 32,747,956.01$
Subject 2 Eigenfaces ($i = 2$)	$s_{21} = 29,325,280.87$	$s_{22} = 1,642,939.20$

2.2 Explanation of Recognition Logic:

The principle of Eigenface recognition is the **minimization of projection residual**. Because a subject's calculated eigenfaces represent their specific, learned visual "grammar" (their unique facial geometry and typical lighting responses), a test image will project much more accurately onto the eigenfaces of the person it actually belongs to. To classify a test image, we compare its projection residuals across all available subject subspaces and assign it to the class with the **lowest score**.

- **For Test Image 1:** We compare s_{11} against s_{21} . Because $5,251,744 < 29,325,280$, the algorithm successfully recognizes the image as Subject 1.
- **For Test Image 2:** We compare s_{22} against s_{12} . Because $1,642,939 < 32,747,956$, the algorithm successfully recognizes the image as Subject 2.

3. Algorithm Performance and Improvements (5 points)

3.1 Performance Evaluation:

The face recognition algorithm works extremely well for this specific test case. The diagonal scores (s_{11} and s_{22}) are roughly an order of magnitude smaller than the cross-subject scores (s_{12} and s_{21}). This massive separation in the residual magnitudes indicates that the 6-dimensional subspaces successfully isolated the unique features of each subject, allowing for unambiguous and correct classification. While successful here, the baseline PCA Eigenface algorithm is highly sensitive to external variables. To improve its robustness for real-world or more complex datasets, I would implement the following improvements:

- **Increase the Number of Principal Components (k):** Using only the top 6 eigenfaces discards a significant amount of structural information. Increasing k would allow the model to capture finer facial geometries beyond just the dominant lighting shadows, improving accuracy on larger datasets.
- **Implement Linear Discriminant Analysis (LDA / Fisherfaces):** PCA maximizes **total** variance, which often means it prioritizes lighting changes over facial structure. Switching to LDA (Fisherfaces) would be a major improvement, as LDA explicitly maximizes the distance **between** different subjects while minimizing the variance **within** a single subject's photos.
- **Lighting Normalization Preprocessing:** Because the top eigenfaces are heavily polluted by room lighting variations, applying a preprocessing step like **Histogram Equalization** to the grayscale images before computing the PCA would force the algorithm to focus more strictly on the physical facial geometry rather than shadow intensity.

5 Correlated Features (motivation for PCA)

1. Co-variance Matrix Rank and Expression

1.1 Construct the Design Matrix X

We are given the design matrix $X = [\mathbf{1}^T; \mathbf{x}_1^T; \mathbf{x}_2^T] \in \mathbb{R}^{3 \times m}$ and $(x_{1,i} = 1.5x_{2,i})$ as the relationship. Because the features are perfectly correlated, we can substitute x_1 with $1.5x_2$:

$$X = \begin{bmatrix} 1 & 1 & \dots & 1 \\ 1.5x_{2,1} & 1.5x_{2,2} & \dots & 1.5x_{2,m} \\ x_{2,1} & x_{2,2} & \dots & x_{2,m} \end{bmatrix}$$

1.2 Compute XX^T

To find the G matrix XX^T , we multiply X by its transpose. This results in a 3×3 matrix containing the dot products of the rows:

$$XX^T = \begin{bmatrix} \sum_{i=1}^m 1 & \sum_{i=1}^m 1.5x_{2,i} & \sum_{i=1}^m x_{2,i} \\ \sum_{i=1}^m 1.5x_{2,i} & \sum_{i=1}^m (1.5x_{2,i})^2 & \sum_{i=1}^m (1.5x_{2,i})(x_{2,i}) \\ \sum_{i=1}^m x_{2,i} & \sum_{i=1}^m (x_{2,i})(1.5x_{2,i}) & \sum_{i=1}^m x_{2,i}^2 \end{bmatrix}$$

we can now substitute the following given in the problem: $m = \sum_{i=1}^m 1$, $T = \sum_{i=1}^m x_{2,i}$, and $S = \sum_{i=1}^m x_{2,i}^2$:

$$XX^T = \begin{bmatrix} m & 1.5T & T \\ 1.5T & 2.25S & 1.5S \\ T & 1.5S & S \end{bmatrix}$$

1.3 Express G

Divide the entire matrix by m to get $G = \frac{1}{m}XX^T$:

$$G = \begin{bmatrix} 1 & 1.5\frac{T}{m} & \frac{T}{m} \\ 1.5\frac{T}{m} & 2.25\frac{S}{m} & 1.5\frac{S}{m} \\ \frac{T}{m} & 1.5\frac{S}{m} & \frac{S}{m} \end{bmatrix}$$

1.4 Proof for $\text{rank}(G) = 2$

The rank of a matrix is the number of linearly independent rows. Let's look at the rows of G :

- **Row 2:** $[1.5\frac{T}{m} \quad 2.25\frac{S}{m} \quad 1.5\frac{S}{m}]$
- **Row 3:** $[\frac{T}{m} \quad 1.5\frac{S}{m} \quad \frac{S}{m}]$

Notice that if you multiply **Row 3** by a scalar of 1.5, you get exactly **Row 2**:

$$1.5 \times [\frac{T}{m} \quad 1.5\frac{S}{m} \quad \frac{S}{m}] = [1.5\frac{T}{m} \quad 2.25\frac{S}{m} \quad 1.5\frac{S}{m}]$$

Because Row 2 is perfectly linearly dependent on Row 3, it contains no unique information. The matrix collapses by one dimension. Row 1 is linearly independent from Row 3. Therefore, the maximum number of linearly independent rows is 2, proving that $\text{rank}(G) = 2$.

2. Largest Eigenvector of G

Because the problem states the features are "standardized by subtracting the means," the sum of the features must be zero. Therefore, $T = \sum x_{2,i} = 0$.

Hence, we can substitute $T = 0$ into the expression for G to simplify it before finding the largest eigenvector.

$$G = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 2.25\frac{S}{m} & 1.5\frac{S}{m} \\ 0 & 1.5\frac{S}{m} & \frac{S}{m} \end{bmatrix}$$

Instead of blindly applying the characteristic equation to find the eigenvalues, we can use our intuition about the structure of this matrix to find the largest eigenvector. We see the special relationship: 2.25 is exactly 1.5^2 . This means that the second and third rows are perfectly correlated, which implies that the largest eigenvector will be a linear combination of these two rows. This also implies that the entire matrix can be perfectly factored into the outer product of a single column vector. To find the largest eigenvector, we look at the 2×2 feature sub-matrix (let's call it G_{sub}):

$$G_{\text{sub}} = \frac{S}{m} \begin{bmatrix} 2.25 & 1.5 \\ 1.5 & 1 \end{bmatrix}$$

We set up the eigenvalue equation $G_{\text{sub}}v = \lambda v$. Because the determinant of this sub-matrix is 0 ($(2.25 \times 1) - (1.5 \times 1.5) = 0$), one eigenvalue is 0. The other eigenvalue is the trace (the sum of the diagonal):

$$\lambda_{\text{max}} = (2.25 + 1)\frac{S}{m} = 3.25\frac{S}{m}$$

Now, substituting $\lambda_{\max}$ back in to find the eigenvector $v = [v_1, v_2]^T$:

$$\frac{S}{m} \begin{bmatrix} 2.25 & 1.5 \\ 1.5 & 1 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = 3.25 \frac{S}{m} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix}$$

We then simplify the second row's equation:

$$1.5v_1 + 1v_2 = 3.25v_2$$

$$1.5v_1 = 2.25v_2$$

$$v_1 = 1.5v_2$$

If we choose $v_2 = 1$, then $v_1 = 1.5$. Mapping this back to the full 3×3 matrix G (where the intercept dimension is unaffected since $T = 0$), the leading eigenvector is:

$$v = \begin{bmatrix} 0 \\ 1.5 \\ 1 \end{bmatrix}$$

3. Interpretation of the Largest Eigenvector

The leading eigenvector $[0, 1.5, 1]^T$ tells us exactly how to optimally combine our features into a single, highly informative metric.

- The **0** in the first position means the intercept (mean) plays no role in the variance, which makes sense because the data is already centered at zero.
- The **1.5** and **1** dictate that the new "principal" feature should be constructed by adding $1.5 \times (\text{Bedrooms}) + 1 \times (\text{Bathrooms})$.

Because the two features are perfectly correlated ($x_1 = 1.5x_2$), every single house in your dataset lies perfectly on a single straight 1D line in the 2D feature space. There is zero scatter or variance off of this line. This leading eigenvector points exactly down that line. It captures 100% of the dominant variation because there is simply no variation anywhere else. By projecting the data onto this eigenvector (performing PCA), you can reduce your model from using two redundant features down to one single "Bed/Bath Size" feature without losing a single drop of information, effectively solving the multicollinearity problem for your linear regression.

6 PCA and Data Centering

The quick short answer is **No, they are not equal in general**. ($w^1 \neq \tilde{w}^1$). The Prof. X's proposal will fail to find the true direction of variance unless the data happens to already be perfectly centered around the origin. Below is the mathematical proof demonstrating exactly why they diverge, followed by the geometric intuition.

We start with relating the two matrices C and $\tilde{C}$ and find the mathematical relationship between them. We start with the definition of $\tilde{C}$:

$$\tilde{C} = \frac{1}{m} \sum_{i=1}^m x^i (x^i)^T$$

Rewriting x^i by explicitly injecting the mean μ , we can write $x^i = (x^i - \mu) + \mu$. Substitute this into the equation:

$$\tilde{C} = \frac{1}{m} \sum_{i=1}^m [(x^i - \mu) + \mu] [(x^i - \mu) + \mu]^T$$

Now, we expand the multiplication of the two terms in the brackets. This gives us four terms:

$$\tilde{C} = \frac{1}{m} \sum_{i=1}^m [(x^i - \mu)(x^i - \mu)^T + (x^i - \mu)\mu^T + \mu(x^i - \mu)^T + \mu\mu^T]$$

We can distribute the summation $\frac{1}{m} \sum$ to each of these four terms. Let's look at the middle "cross terms." Because the sum of deviations from the mean is always zero (as we established in your previous problem), these terms vanish!

$$\frac{1}{m} \sum_{i=1}^m (x^i - \mu)\mu^T = \left(\frac{1}{m} \sum_{i=1}^m x^i - \frac{1}{m} \sum_{i=1}^m \mu \right) \mu^T = (\mu - \mu)\mu^T = 0$$

This leaves us with only the first and last terms:

$$\tilde{C} = \left[\frac{1}{m} \sum_{i=1}^m (x^i - \mu)(x^i - \mu)^T \right] + \left[\frac{1}{m} \sum_{i=1}^m \mu\mu^T \right]$$

The first bracketed term is exactly the definition of the centered covariance matrix C . The second bracketed term simplifies to just $\mu\mu^T$ (since summing a constant m times and dividing by m changes nothing).

This gives us our golden relationship:

$$\tilde{C} = C + \mu\mu^T$$

Now that we know $\tilde{C} = C + \mu\mu^T$, let's test if they share the same leading eigenvector.

By definition, w^1 is the eigenvector of C , meaning:

$$Cw^1 = \lambda_1 w^1$$

If our initial guess was correct that they are equal and the eigenvectors are the same, then applying $\tilde{C}$ to w^1 should also result in a scaled version of w^1 . Looking at the below expression

$$\begin{aligned} \tilde{C}w^1 &= (C + \mu\mu^T)w^1 \\ \tilde{C}w^1 &= Cw^1 + \mu\mu^T w^1 \end{aligned}$$

Substitute Cw^1 with $\lambda_1 w^1$:

$$\tilde{C}w^1 = \lambda_1 w^1 + \mu(\mu^T w^1)$$

Notice the term on the far right: $\mu(\mu^T w^1)$. Because $(\mu^T w^1)$ is just a scalar number (a dot product), this term is a vector pointing in the exact direction of the mean μ .

For $\tilde{C}w^1$ to be an eigenvector, the entire right side of the equation must point in the same direction as w^1 . But we are adding a vector that points in the direction of μ . In conclusion, unless the mean happens to lie perfectly on the principal axis (so μ is parallel to w^1), or the data is already centered ($\mu = 0$), the addition of the mean vector pulls the result off-axis. Therefore, $\tilde{C}w^1$ is not parallel to w^1 , which proves that w^1 is not an eigenvector of $\tilde{C}$.

The Geometric Intuition: Beyond the math which was proven above, the assumption that the eigenvectors are the same is incorrect and how to interpret in the real world.

- **Standard PCA (C):** Subtracting the mean moves the data to the origin. This allows PCA to find the axis of **maximum variance** (how the data is spread out relative to itself).

- **Uncentered PCA ($\tilde{C}$):** If the data is sitting far away from the origin (e.g., a cluster of points centered at coordinates $[10, 10]$), the uncentered matrix doesn't look at internal variance. It just looks for the axis of maximum magnitude from the origin. The leading eigenvector will just become a giant arrow pointing straight from $(0, 0)$ to the center of the cluster, completely ignoring the actual shape or spread of the data.